vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases

Angel Coronel

Crypto Protocols – CUNY

12/18/25

Problem: How can a weak client query an untrusted server and efficiently verify the answer is correct *without downloading the whole DB*?

Motivation

- **Outsourcing is standard:** databases live in the cloud; clients are often resource-limited
- **But the server is untrusted:** it may return wrong answers (bugs, misconfigurations, or malicious behavior)
- **Goal:** verify correctness of a SQL query result *without* re-running the query or storing the database locally
- **Want:**
  - **Succinct client verification** (small work/state)
  - **Small communication** (proof smaller than DB)
  - **Dynamic DB support** (updates)
  - **Expressive queries** (not just a narrow class)

vSQL gives a weak client a succinct proof that the SQL result was computed correctly over a committed outsourced database state.

High-Level Architecture (Client–Server)

- **Commit:** client stores a small commitment (digest) to DB
- **Query:** server returns (answer, proof) for SQL query on its DB
- **Verify:** client checks proof using only digest and randomness
- **Update:** server proves updates so digest evolves correctly

key idea

CMT protocol verifies a *query circuit*. **polynomial delegation** lets the client verify the (data-dependent) *evaluations* CMT needs without storing the DB

Client (weak verifier)	**Server (prover)**
Stores commitment/keys	Stores full database
Sends SQL query Q	Computes answer *ans*
Samples verifier randomness	Produces proof Π
Verifies (ans, Π) fast	Returns (ans, Π)

Roadmap

1. **CMT interactive proof**
2. **Multivariate polynomials and delegation**
3. **vSQL:** commit, query, verify, update
4. **SQL $\rightarrow$ circuit mapping:** selection, join, aggregation/group-by
5. **Small example**
6. **Summary**

Arbitrary SQL Queries

Example SQL query

```
SELECT c.tier, SUM(o.price) AS total_spent
FROM Orders o
JOIN Customers c ON o.cust_id = c.cust_id
WHERE o.status = 'PAID'
GROUP BY c.tier
HAVING SUM(o.price) > 100;
```

- **WHERE:** row-level predicates (filters)
- **JOIN:** relate multiple tables by key equality
- **GROUP BY:** partition rows into groups
- **Aggregates:** `SUM`, `COUNT`, `AVG`, etc.
- **HAVING:** predicates on aggregated values

vSQL must verify the correctness of *any such query* over a large outsourced database, without the client storing or recomputing the data

Cormode, Mitzenmacher, and Thaler (CMT) Interactive Proof System

- It is a general-purpose interactive proof system designed for verifying the correctness of computations with high efficiency for the verifier and a significantly reduced runtime for the prover compared to earlier constructions
  - based on the GKR protocol framework
- CMT verifies evaluation of a **layered arithmetic circuit** C over a field $\mathbb{F}$
- Prover claims $y = C(x)$.
- Verifier wants accept iff true, while doing *much less work* than evaluating C

CMT reduces *circuit correctness* to a sequence of **low-degree polynomial identities** checked at **random points** (like using sumcheck reductions). SQL queries can be compiled into circuits; if we can verify the circuit, we can verify the query logic

CMT Protocol Skeleton

- Circuit is layered: input layer $\rightarrow$ intermediate layers $\rightarrow$ output
- Start from claimed output value and work **backwards** layer-by-layer
- At each layer:
  1. Encode "this layer is consistent with previous layer" as a **low-degree identity**
  2. Use a **random challenge** to reduce a global claim to a **small number of point checks**
  3. Move the claim to the previous layer

soundness intuition

if prover lies, it must satisfy a false low-degree identity; a random point catches this with high probability

What CMT Needs for vSQL

- CMT ends by requiring the verifier to check **evaluations of input-layer polynomials** at random points chosen by the verifier
- In the classic setting, verifier can compute these because it **has the input** x
- In outsourced DB setting, the client **does not have/store** x

a bottleneck:

CMT reduces verification to "evaluate a data-dependent low-degree polynomial at a random point." **but the client has no data**

We need a way to *delegate* these multivariate polynomial evaluations to the server *with proofs*

Multivariate Polynomial Delegation

- The verifier must check values like $f(r)$ where f is derived from the database
- The verifier chooses r randomly; the server must answer with a value and a proof
- Want to **commit once** then **open at many random points** over time

Goal

Be able to have verifiable access to polynomials based on the DB *without* downloading or storing the DB

From Tables to Multivariate Polynomials: Multilinear Extension (MLE)

- View a column (length N) as a function on Boolean indices:

$$a : \{0,1\}^\ell \to \mathbb{F}, \qquad N = 2^\ell$$

- Extend it to a polynomial on the whole field:

$$\tilde{a} : \mathbb{F}^\ell \to \mathbb{F}$$

  called the **multilinear extension (MLE)**
- Key properties:
  - $\tilde{a}(b) = a(b)$ for all $b \in \{0,1\}^\ell$
  - degree 1 in each variable (multilinear)
  - "sum over rows" becomes "sum over Boolean hypercube" (good for sumcheck)

Intuition

A random evaluation $\tilde{a}(r)$ acts like a compact fingerprint of the entire column

Polynomial Delegation

We want a commitment / delegation primitive for a (multi-variate) polynomial f:

- **Commit:** client computes a commitment *com* to f (stored as the digest)
- **Open:** on verifier's chosen r, the server returns $y = f(r)$ and proof π
- **Verify:** client checks $\text{Verify}(com, r, y, \pi) = 1$

(informal) required guarantees

- **Soundness/binding:** cannot open same commitment to two different values at the same r
- **Succinctness:** proofs and verification are small/fast
- **Knowledge property:** accepting implies prover is consistent with an underlying polynomial/witness

Concrete Delegation in vSQL

- vSQL's delegation layer is **pairing-based** (commitments + succinct opening proofs)
- Treat it as a black box (for this presentation) with the property:

$$\text{Verify}(com, r, y, \pi) = 1 \;\Rightarrow\; y = f(r) \text{ for the committed polynomial } f$$

- This is what lets the client trust evaluations on the database inside the CMT verification

Delegation is what replaces "verifier computes input evaluations" with "server proves them"

Bridge: How CMT and Delegation Become vSQL

- **CMT** proves the query circuit C_Q outputs the claimed answer *ans*
- CMT reduces correctness to checking evaluations of **input-layer polynomials** at random points
- **Polynomail delegation** proves those evaluations are correct with respect to the committed DB

$$\underbrace{\text{CMT proof (verifies query logic)}}_{\text{circuit correctness}} + \underbrace{\text{Eval proofs (verifies data dependence)}}_{\text{committed DB}} \Rightarrow \textbf{vSQL verification}$$

vSQL (Commit, Query, Verify, Update)

1. **Setup/KeyGen:** client creates public parameters (and small secret state as needed)
2. **Commit DB:** server represents relations/columns as MLE polynomials and sends commitments; client stores digest
3. **Query:** client sends SQL query Q
4. **Prove:** server returns *ans* and proof Π:
   - CMT transcript for circuit C_Q (proving circuit correctness)
   - polynomial delegation openings for required MLE/witness evaluations
5. **Verify:** client checks Π and outputs *ans* if its valid
6. **Update:** server updates tables and provides an update proof so the digest evolves correctly

Security & Assumptions

- **Threat:** malicious server tries to return an incorrect answer with an accepting proof
- **Goal (informal soundness):** if client accepts, answer is correct for the *committed DB* except withnegligible probability
- **Assumptions:**
  - standard hardness assumptions in bilinear groups for binding/sound openings (q-SDH)
  - a knowledge assumption used in the paper to argue extractability/consistency for delegated/witness components

Server can't forge evaluations inconsistent with the committed database state (and cannot cheat via inconsistent witnesses)

SQL to Circuit Mapping

- vSQL compiles a SQL query into a **layered arithmetic circuit** C_Q
- Inputs to C_Q are:
  - committed DB columns (as MLE polynomials)
  - auxiliary values (witness) given by prover when needed
- The prover claims *ans* and proves: $C_Q(\text{DB}, \text{witness}) = \textit{ans}$.

intuition

relational operators become arithmetic "gadgets"; CMT verifies the gadget composition; delegation ties all values back to the committed DB

"Gadgets" are performance optimizations using how well the CMT protocol and multivariate polynomials go hand in hand with SQL to make some SQL common queries efficient

Selections/WHERE Predicates

- For each row i, compute a **mask bit** $b_i \in \{0, 1\}$ indicating if the predicate holds
- In arithmetic circuits, AND is multiplication:

$$b_i = \prod_{t=1}^{T} \text{Pred}_t(i)$$

  where each $\text{Pred}_t(i) \in \{0, 1\}$ is a sub-predicate check
- Use the mask to "filter" values:

$$\text{filtered_val}_i = b_i \cdot v_i$$

circuit-friendly

Filtering becomes multiplication by a boolean indicator

Aggregations (SUM/COUNT)

- After computing masks b_i, aggregations are simple field sums
- **SUM over a column v:**

$$\text{SUM} = \sum_i b_i \cdot v_i$$

- **COUNT of selected rows:**

$$\text{COUNT} = \sum_i b_i$$

- **AVG:** typically computed as SUM/COUNT with an added constraint that COUNT $\neq 0$ (or handle the empty case explicitly)

Most SQL aggregates reduce to linear algebra once you have the indicator masks

Joins (and witness)

- Naively, a join checks all row pairs into a quadratic-size circuit (too big)
- vSQL uses a **witness-based join**:
  - prover supplies, for each left-row i, an index $j(i)$ of the matching right-row
  - circuit checks the join keys match (e.g., $key_L(i) = key_R(j(i))$)
- This moves the expensive "search for matches" outside the circuit,and keeps the circuit nearly linear in table size

soundness intuition

A bogus mapping $j(i)$ is caught because the circuit enforces key equality (and any multiplicity/consistency constraints required by the query)

GROUP BY

- GROUP BY partitions rows into buckets by a key (like customer_id)
- Circuit view: create **bucket indicator masks**

$$b_{i,g} \in \{0,1\} \quad \text{meaning "row } i \text{ belongs to group } g \text{ (conceptually)"}$$

- Then aggregate per group:

$$\text{SUM}_g = \sum_i b_{i,g} \cdot v_i \qquad \text{and} \qquad \text{COUNT}_g = \sum_i b_{i,g}$$

- In practice, vSQL structures this so the circuit remains efficient (no huge $i \times g$ blowup)

GROUP BY is "many masked sums". the hard part is representing groups efficiently

Putting Operators Together: Query $\Rightarrow$ Circuit

- A realistic SQL query is a pipeline of operators:

$$(\text{Scan}) \rightarrow (\text{Filter}) \rightarrow (\text{Join}) \rightarrow (\text{Aggregate})$$

- vSQL composes the corresponding gadgets into one layered circuit C_Q
- The prover supplies:
  - the query answer *ans*
  - witness values needed by gadgets (e.g., join mapping, intermediate masks)
- CMT then proves the circuit output equals *ans*

Example Query (Toy) + Ground Truth

Table: Orders(order_id, status, price)

order_id	status	price
1	PAID	10
2	PENDING	7
3	PAID	5
4	PAID	2

Query:

```
SELECT SUM(price) FROM Orders WHERE status = 'PAID';
```

Ground truth

Mask $b = [1, 0, 1, 1]$ so SUM $= 10 + 5 + 2 = 17$.

Example as a Circuit: Mask then Aggregate

- For each row i:

$$b_i = [\text{status}_i = \texttt{PAID}] \in \{0, 1\} \qquad \text{filtered}_i = b_i \cdot \text{price}_i$$

- Output:

$$ans = \sum_i \text{filtered}_i$$

- Prover returns:
  - claimed answer $ans = 17$
  - a proof that the circuit output is 17 on the committed table state

How does the verifier access status and price values without storing the table?

Example Verification: Random Points, Openings & CMT Closes

- Encode columns as MLE polynomials over row indices:

$$\widetilde{\text{price}} : \mathbb{F}^2 \to \mathbb{F}, \quad \widetilde{\text{status}} : \mathbb{F}^2 \to \mathbb{F}$$

  and client stores commitments $com(\widetilde{\text{price}})$, $com(\widetilde{\text{status}})$.
- During CMT, verifier samples random points $r \in \mathbb{F}^2$ and needs evaluations like $\widetilde{\text{price}}(r)$ and $\widetilde{\text{status}}(r)$
- Server provides each needed evaluation with a delegation proof & verifier checks them
- With these verified evaluations, verifier completes the CMT checks and accepts/rejects

If verifier accepts, then the claimed *ans* equals the true SQL answer on the committed Orders table state.

Performance Analysis

- The paper evaluates vSQL on standard decision-support style SQL workloads (TPC-H style).
- Key qualitative takeaways reported:
  - **Client verification is small/fast** relative to query execution
  - **Server proving is costly** (integrity vs. performance trade-off)
  - vSQL aims to avoid query-specific preprocessing and support dynamic updates

vSQL is designed for settings where server overhead is acceptable in exchange for strong correctness guarantees for a weak client

Trade-offs, Limitations, and Discussion

- **Pros:** succinct client verification, small client storage, expressive queries (doing compile to circuit), dynamic updates
- **Cons:** server proving overhead; pairing crypto; stronger assumptions
- **Limitations/open questions:**
  - engineering complexity of query compilation & witness generation
  - supporting more SQL features efficiently (complex joins, nested queries, string ops)
  - reducing prover overhead (and comparison to newer proof systems)

Summary

- **CMT:** efficiently verifies layered arithmetic circuits by doing low-degree checks at random points
- **MLE:** makes relational data "polynomial friendly" and compatible with sumcheck verification
- **Polynomial Delegation:** lets a weak verifier check data-dependent evaluations without storing the DB
- **vSQL:** composes CMT (verify computation) plus delegation (verify data) to verify outsourced SQL queries and updates
- **Joins:** made practical with witness strategies (prover searches; circuit checks constraints)

vSQL turns SQL verification into **circuit verification** plus **verifiable access to committed data**

thank you!

Questions ?